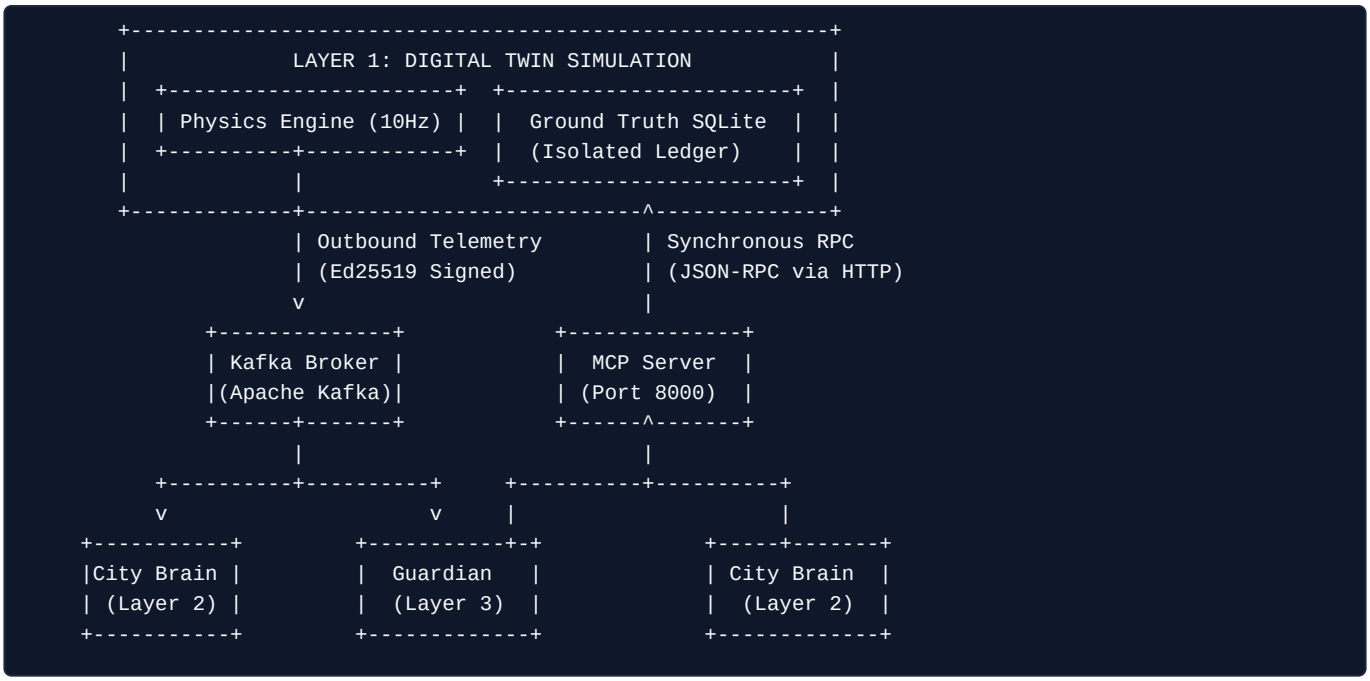
Twin Layer: Developer & Integration Guide

This guide walks you through setting up, running, testing, and integrating with Layer 1 (Twin Layer). Whether you are developing Layer 2 (City Brain), Layer 3 (Guardian), or contributing to Layer 1, this document provides the exact commands, code snippets, network parameters, and architectural contracts you need.

---

1. System Overview & Architecture

Layer 1 simulates 42 physical IoT devices across 6 geographic zones in Greater Tunis. It runs a deterministic 10 Hz simulation loop (100 ms fixed timestep) driven by diurnal mathematical curves (traffic cycles, water drainage kinetics, electrical loads, particulate diffusion, and Poisson emergency call distributions).



Core Architecture Rules

1. **Unidirectional Streaming on Kafka:** Twin Layer streams telemetry to the Kafka topic `twin.sensors`. Partners only consume from this topic; never publish data to it. The topic must already exist on the broker (the Twin never creates topics or modifies broker settings). Messages are keyed by `device_id` to maintain strict per-device ordering within partitions.
2. **Durable Producer Configuration:** Outbound streaming uses `acks="all"` and `enable.idempotence=True` by default to prevent message loss across multi-node broker clusters.
3. Configurable via `KAFKA_ACKS`: `"all"` (default, enables idempotence), `"1"` (leader only, disables idempotence), or `"0"` (fire-and-forget). Any other value raises a startup error.
4. Multi-broker clusters: `KAFKA_BOOTSTRAP_SERVERS` supports comma-separated addresses (`host1:9092,host2:9092,host3:9092`).
5. Custom TLS certificates: `KAFKA_SSL_CA_LOCATION` points to a CA certificate bundle for `SASL_SSL` brokers.
6. Pre-flight connection: Set `WAIT_FOR_KAFKA=false` if partner ACLs restrict broker metadata queries.
7. **Synchronous Commands over MCP:** Actuation commands (`dispatch`, `set_valve`, etc.) and queries (`get_device_key`, `list_devices`) run strictly over the Model Context Protocol (MCP) HTTP/SSE interface on port `8000`. We do not send commands over Kafka.
8. **Dual Clocks:**
9. **Simulation Time (`ts`):** RFC 3339 UTC string with second precision ending in `'Z'` (e.g. `2026-10-18T14:00:00Z`). All sensor timestamps reflect simulation time.

10. **Wall-Clock Time:** Used only inside the Twin process to pace the 10 Hz loop (sleeping ~100 ms between ticks).
11. **Sub-300-Byte Payload Budget:** Every JSON payload pushed to Kafka stays strictly <= 300 bytes.
12. **Cryptographic Signatures:** Every reading is signed by the device's private Ed25519 key. The signature is placed in the `sig` field of the Kafka envelope.
13. **Ground Truth Isolation:** The local SQLite database (`ground_truth.db`) records true physical values and injected attacks. Neither City Brain nor Guardian can query this file directly. They must rely on public MCP endpoints and Kafka messages.

---

2. Secrets Management & Zero-Downtime Reload

The Twin provides a pluggable secrets resolver and hot-reloading mechanism for runtime secrets:
- `KAFKA_PASSWORD` : SASL authentication credential.
- `MCP_AUTH_TOKENS` : Principal bearer tokens (`principal:token,...`).
- `RED_TEAM_TOKEN` : Authentication token for the Red Team REST API.

Note: Device signing keys located at `KEYSTORE_PATH` are long-term identity keys and are not runtime secrets. They do not rotate at runtime.

Secret Resolution Order

For any secret `NAME` , the resolution precedence is:
1. `TWIN_SECRET_HOOK` : Pluggable hook in `module:func` format. If configured, the function is invoked as `func(name) -> Optional[str]` . This allows clean integration with HashiCorp Vault, AWS Secrets Manager, or custom key stores without adding external dependencies.
2. `<NAME>_FILE` : File path containing the secret (e.g. `KAFKA_PASSWORD_FILE` , `RED_TEAM_TOKEN_FILE` , `MCP_AUTH_TOKENS_FILE`), suitable for Docker secrets, Kubernetes secrets, or Vault Agent sinks.
3. `<NAME>` : Standard environment variable.

Hot Reloading Without Downtime

Secrets can be refreshed at runtime without restarting the simulation loop:
- **REST endpoint:** `POST http://localhost:8100/api/secrets/reload` (authenticated with current `X-Red-Team-Token`).
- **Signal handler:** `kill -HUP <twin_pid>` .
- **Periodic background polling:** Configured via `TWIN_SECRET_RELOAD_SECONDS=<seconds>` .

Credential Probe Before Swap

When a new Kafka password is provided during reload, the Twin executes a safety probe before swapping credentials:
1. A candidate producer is initialized with the candidate password.
2. A metadata query probe (`list_topics(timeout=5.0)`) verifies that the broker accepts the new credentials.
3. If the check succeeds, the candidate producer is swapped in and the old producer is drained.
4. If the check fails or times out, the candidate producer is immediately discarded, the existing producer remains active without dropping messages, and the reload endpoint reports an error without leaking secret values.

HashiCorp Vault Communication Key Synchronization

The Twin Layer can optionally synchronize dynamic inter-layer communication keys directly from HashiCorp Vault KV v2 via AppRole authentication:
- **Paths synchronized:**
- `comm/twin/guardian/private`
- `comm/twin/brain/private`
- `comm/guardian/twin/public`
- `comm/brain/twin/public`
- **Configuration:** Set `VAULT_ADDR="https://<cluster>.hashicorp.cloud:8200"` , `VAULT_ROLE_ID_FILE=/app/secrets/vault_twin_role_id` , and `VAULT_SECRET_ID_FILE=/app/secrets/vault_twin_secret_id` (host: `secrets/...`).

- **Formats supported:**
- `pem-key` : Standard PEM payload format `{"key": "<PEM>"}` .
- `kid` : Future rotating format `{"kid": "...", "private_key"|"public_key": "<PEM>", "fingerprint": "<sha256 hex>", "active_after": "..."}` .
- **Algorithm detection:** Automatically detects key types via standard `cryptography` primitives (`X25519` , `Ed25519` , `RSA-<bits>`).
- **Rotation discipline:** For `kid` format private keys, if `active_after` is in the future, the Twin uses `version-1` as the active key. For public keys, both latest and previous versions are retained in memory.
- **Traffic usage note:** Communication keys are fetched and refreshed in background memory, but not yet used for traffic encryption.
- **Inspection:** Live metadata (path, version, algorithm, kid, last_refresh_wall, last_error) is exposed at `GET /api/status` under the `vault` key.

---

3. Containment Persistence Across Restarts

Device containment enforced by Layer 3 (Guardian) persists across process restarts:
- When Guardian calls `isolate_device` or `quarantine_device` , the device's operational status in SQLite table `devices.status` is updated in the same atomic transaction as the `containment_log` audit record.
- Rogue device identifiers (hardware not registered in `devices`) are recorded in `containment_log` .
- Upon simulator startup, the Twin reads `devices.status` and `containment_log` to restore all device isolation states and blocked rogue IDs before the first simulation tick executes.
- Devices remain contained until an authorized principal invokes `reinstate_device` or `release_device` .

---

4. Adversary Delay Backlog & Visibility

When an adversary activates the `link` capability in `delay` mode, frames for the targeted sensor are buffered in memory:
- **Queue visibility:** The Red Team status endpoint (`GET http://localhost:8100/api/status`) exposes `delayed_frames_pending` , indicating the total number of frames currently buffered across all delay queues.
- **Immediate FIFO flush:** When a delay attack is deactivated (via `/api/attack/clear` , individual attack clear, or simulator settle), all held frames are emitted immediately to the Kafka wire in chronological FIFO order, and `delayed_frames_pending` drops to 0.

---

5. Scenario Settle Step & Testing Discipline

Automated scenarios in `scenarios/` employ a deterministic settle procedure to ensure test independence and consecutive run stability:
1. **Clear adversary state:** Clears all active attacks and physical events via the Red Team API.
2. **Drain delay backlog:** Waits until `delayed_frames_pending == 0` .
3. **Reset actuators:** Restores traffic cameras to `normal` and power loads to `grid_a` .
4. **Reinstate devices:** Calls `reinstate_device` as caller `scenario` for any isolated or quarantined devices.
5. **Drain flood water:** Polls water level sensors until readings drop below 5.0 m (temporarily commanding drainage valves to `open` , then restoring them to `half`).
6. **Verify quiet baseline:** Confirms that no new containment or command events appear in the journal for at least 10 simulation seconds.
7. **Attribution boundary:** Scenario false-positive guards filter out journal entries prior to settle completion (`sim_ts <= settle_end_sim_time`), preventing cross-scenario false alarms.

---

6. Mock Lifecycle & Process Management

Live mock implementations (`twin.mocks.mock_brain` , `twin.mocks.mock_guardian`) feature safe lifecycle management:
- `--exit-with-parent` : When launched by scenario runners or parent processes, mocks continuously monitor parent process liveness and terminate cleanly if the parent exits.
- Signal traps: Standard `SIGINT` and `SIGTERM` handlers ensure orderly consumer disconnects and socket closures.

Mock Environment Variables

The standalone mock processes support the following configuration environment variables:

Variable	Default	Description
`TWIN_MCP_URL`	`http://localhost:8000`	Twin MCP server endpoint URL for tool calls (`--mcp-url`).
`MCP_AUTH_TOKEN`	*(unset)*	Optional bearer token passed in `Authorization: Bearer <token>` for MCP authentication.
`KAFKA_BOOTSTRAP`	`localhost:9092`	Kafka bootstrap broker address for consuming telemetry (`--kafka-bootstrap`).
`BRAIN_LOG_PATH`	*(unset)*	Destination file path for City Brain mock structured execution log entries.
`GUARDIAN_LOG_PATH`	*(unset)*	Destination file path for Guardian mock structured execution log entries.

---

7. Quickstart: Running the System

Docker Compose

```
# Start all services with accelerated simulation speed:
SIM_SPEED=20 docker compose up -d --build

# Retrieve the red-team token generated at container boot:
export RED_TEAM_TOKEN=$(docker exec twin_layer cat /app/data/red_team.token)

# View running container status:
docker compose ps
```

Local Development (No Docker)

```
python3 -m venv venv
source venv/bin/activate
pip install -r twin/requirements.txt

# Start the Twin runtime:
python3 -m twin.main
```

---

8. Complete MCP Tool Reference

All tools are accessible via `http://localhost:8000/sse` and `/mcp` using standard MCP JSON-RPC format.

Queries

Tool	Parameters	Returns	Description
`list_devices`	*(none)*	`list[dict]`	Returns all 42 registered sensors, types, zones, and status.
`get_reading`	`device_id` (str)	`dict`	Returns the latest broadcast reading for a specific device.
`get_readings`	`device_id` (str), `limit` (int), `since_ts` (str, optional)	`list[dict]`	Returns chronological historical readings with optional RFC 3339 filter.
`get_device_state`	`device_id` (str)	`dict`	Returns current operational status, isolation state, and active actuation.
`get_city_state`	*(none)*	`dict`	Returns zone-by-zone aggregate metrics across Greater Tunis.
`get_device_key`	`device_id` (str)	`dict`	Returns Ed25519 public key hex string and key status (`active` / `revoked`).

Actuations (City Brain Controls)

Tool	Parameters	Allowed Target States	Description
`dispatch`	`incident_type` (str), `zone_id` (str)	`medical`, `fire`, `police`, `hazmat`	Dispatches emergency units; reduces active incident count.
`set_signal_plan`	`device_id` (str), `target_state` (str)	`green_wave`, `all_red`, `adaptive`	Controls traffic camera intersections; reduces traffic congestion.
`set_valve`	`device_id` (str), `target_state` (str)	`open`, `half`, `closed`	Controls flood drainage valves; lowers flood water level.
`switch_grid`	`device_id` (str), `target_state` (str)	`grid_a`, `grid_b`, `island`	Reroutes power substation feeder lines.
`get_command_result`	`command_id` (str)	-	Polls execution status of previously submitted command.

Containment (Guardian Controls)

Tool	Parameters	Description
`quarantine_device`	`device_id` (str)	Revokes device network authority; drops future wire transmissions.
`release_device`	`device_id` (str)	Restores network authority after threat clearance.
`isolate_device`	`device_id` (str)	Disconnects device physical subsystem.
`reinstate_device`	`device_id` (str)	Reconnects physical subsystem to nominal status ok.

Tool	Parameters	Description
`rollback_reading`	`reading_id` (str)	Marks poisoned reading as rolled_back in broadcast cache.

---

9. Running Tests

```
# Run unit and conformance tests inside twin directory:
cd twin && python3 -m pytest -q

# Run complete test suite from repository root:
python3 -m pytest -q

# Run all 11 integration scenarios against running Twin:
python3 scenarios/run_all.py --with-mocks
```